

TalkVinyl

Marketplace de Vinilos con Arquitectura Hexagonal

Illyana Julianne Engle Reyes

27 de septiembre de 2026

Índice

1. Introducción	2
1.1. Objetivos	2
2. Arquitectura del Sistema	2
2.1. Capas	2
2.2. Diagrama	2
3. Modelo de Base de Datos	3
3.1. Tabla usuarios	3
3.2. Tabla productos	3
3.3. Tabla pedidos	3
4. Endpoints Principales	4
4.1. Autenticación	4
4.2. Usuarios (admin)	4
4.3. Productos	4
4.4. Pedidos	4
5. Guía de Despliegue en WSL	4
5.1. Requisitos	4
5.2. Pasos	5
5.3. Credenciales de prueba	5
6. Conclusiones	5

1. Introducción

TalkVinyl es una plataforma de comercio electrónico para venta de vinilos de música. El sistema implementa una **arquitectura hexagonal** (Ports & Adapters) con separación estricta de capas, permitiendo que la lógica de negocio sea independiente de los frameworks y las bases de datos.

1.1. Objetivos

- Modelar tres entidades interconectadas: **Usuario**, **Producto** y **Pedido**.
- Implementar CRUDs completos para cada entidad con control de acceso por roles.
- Asegurar las credenciales mediante hashing con bcrypt.
- Gestionar imágenes de productos fuera de la base de datos.
- Aplicar arquitectura hexagonal en el backend.
- Construir una SPA en React con módulos por rol.

2. Arquitectura del Sistema

2.1. Capas

El backend está organizado en cuatro capas:

1. **Dominio:** entidades puras (**User**, **Product**, **Order**) y puertos de persistencia.
2. **Aplicación:** servicios con la lógica de los casos de uso.
3. **Infraestructura:** adaptadores que implementan los puertos (PostgreSQL, bcrypt, multer, svg-captcha).
4. **Interfaces:** controladores HTTP (Express) y middlewares de autenticación y roles.

2.2. Diagrama

INTERFACES (HTTP)
Controllers + Middlewares

APPLICATION (Servicios)
userService / productService
orderService / captchaStore

DOMAIN
Entidades + Puertos + Reglas

3. Modelo de Base de Datos

3.1. Tabla usuarios

- id SERIAL PRIMARY KEY
- nombre VARCHAR(100)
- email VARCHAR(150) UNIQUE
- password_hash VARCHAR(255)
- rol VARCHAR(20) — admin, producto, pedido o NULL
- estado VARCHAR(20) — pendiente, activo, rechazado
- created_at TIMESTAMP

3.2. Tabla productos

- id SERIAL PRIMARY KEY
- nombre, artista, descripcion
- precio NUMERIC(10,2)
- imagen_url VARCHAR(255) — ruta al archivo (no el binario)
- stock INT
- estado VARCHAR(20) — pendiente, aprobado, rechazado, vendido
- creado_por INT REFERENCES usuarios(id)

3.3. Tabla pedidos

- id SERIAL PRIMARY KEY
- usuario_id, producto_id
- cantidad INT
- total NUMERIC(10,2)
- estado VARCHAR(20)

4. Endpoints Principales

4.1. Autenticación

- POST /usuarios — Registro con captcha
- POST /login — Login con captcha → JWT
- GET /captcha — Genera captcha SVG

4.2. Usuarios (admin)

- GET /usuarios — Listar
- PUT /usuarios/:id/aprobar — Aprobar y asignar rol
- PUT /usuarios/:id/rol — Cambiar rol
- DELETE /usuarios/:id — Eliminar

4.3. Productos

- GET /productos — Listar según rol
- GET /productos/catalogo — Catálogo público
- POST /productos — Crear (multipart con imagen)
- PUT /productos/:id/aprobar — Aprobar
- DELETE /productos/:id — Eliminar

4.4. Pedidos

- POST /pedidos — Crear pedido
- GET /pedidos — Listar según rol
- PUT /pedidos/:id/aprobar — Aprobar pedido
- GET /pedidos/mis-solicitudes — Solicitudes del vendedor

5. Guía de Despliegue en WSL

5.1. Requisitos

- WSL (Ubuntu)
- Node.js 20+
- PostgreSQL 14+
- Git

5.2. Pasos

```
# 1. Clonar
git clone https://github.com/illyanaengle22-ui/Node-JS-Mysql.git webapp
cd webapp

# 2. Base de datos
sudo -u postgres psql -c "CREATE DATABASE practica_api;"
sudo -u postgres psql -d practica_api -f backend/migrations/001
_init_vinilos.sql

# 3. Backend
cd backend && npm install
# Crear src/.env con DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, JWT_SECRET
node src/server.js

# 4. Frontend (nueva terminal)
cd ../frontend && npm install
npm run dev
```

5.3. Credenciales de prueba

Email	Contraseña
admin@vinilos.com	Admin123!

6. Conclusiones

El proyecto demuestra la aplicación práctica de arquitectura hexagonal en un caso real de comercio electrónico, con separación estricta de responsabilidades, seguridad de credenciales y control de acceso por roles. La solución es mantenible, escalable y testeable.